

ROHITH DOCUMENT STUDIO RDS

Unified, Offline-First Document and PDF Workspace

Codex-Ready Product Requirements, Architecture, Tool Catalogue, Build Roadmap and Acceptance Tests

Single Complete Edition

All capabilities included. No free-versus-paid feature separation. No artificial usage limits.

Prepared for V. Sai Rohith
Version 1.0 - July 2026
Print format: A4, black and white

1. Executive Decision

Build one private, Windows-first desktop application that combines document conversion, PDF organization, editing, OCR, security, e-signature, automation and optional AI assistance. The application processes files locally by default and presents every feature in one complete edition.

Decision	Selected direction
Product name	Rohith Document Studio (RDS)
Primary platform	Windows desktop first; macOS/Linux later if useful
Application shell	Tauri 2 with React, TypeScript and Vite
Core processing	Local Rust/Python/CLI adapters; no mandatory cloud upload
PDF viewing	PDF.js-based workspace with thumbnails, search and page selection
Data ownership	Files remain on the device; originals are never overwritten by default
Commercial model	Personal-use single edition; no subscriptions or artificial limits
AI policy	Optional; local models preferred; external API only after explicit consent
Build strategy	Feature-equivalent clean-room implementation, not copied code, assets, text or UI

2. Product Vision and Scope

RDS should feel like a complete document workshop rather than a collection of disconnected buttons. A user opens or drops files once, selects an operation, previews the result, runs the job, verifies the output and saves a new file. The same operation contract, history model and error language must be used throughout the app.

- Fast for routine school and office work: merge, split, compress, convert, OCR, sign, redact and print.
- Safe for sensitive documents: local processing, explicit output location, temporary-file cleanup and audit history.
- Powerful for repeated work: batch jobs, saved recipes, watched folders and command-line automation.
- Extensible for future needs: plugin-style operation adapters and optional AI document intelligence.

3. Product Principles

Principle	Meaning
Local first	The default path never uploads the source document.
Never destroy originals	Every operation writes to a new output unless the user explicitly approves replacement.
Preview before commit	Operations that alter pages, visual content, security or signatures provide a preview or clear summary.
One complete edition	All implemented capabilities are visible and available.
Deterministic operations	The same input and settings should produce materially equivalent output.
Transparent failure	Show which file, page, stage and dependency failed, with a practical recovery action.
Accessible and printable	Keyboard navigation, scalable UI and readable monochrome output are first-class requirements.
Clean-room implementation	Replicate useful functions and workflows, but create original

Principle	Meaning
	code, wording, design and assets.

4. Complete Unified Feature Catalogue

The following catalogue combines capabilities that are often divided between free and premium offerings into one product. Milestone labels indicate implementation order only; they are not access tiers.

Area	Tool	Required behavior	Build
Organize	Merge PDFs	Combine many PDFs or images; drag reorder; page-range import; bookmarks; outline preservation	MVP
Organize	Split PDF	By page ranges, every N pages, file size target, bookmarks, blank pages or selected thumbnails	MVP
Organize	Extract pages	Create one PDF, individual PDFs or ZIP from selected pages	MVP
Organize	Remove pages	Thumbnail selection, ranges and undo before export	MVP
Organize	Reorder pages	Drag-and-drop, reverse, odd/even grouping and rotate while organizing	MVP
Organize	Rotate pages	90/180/270 degrees; selected, odd, even or all pages	MVP
Organize	Scan to PDF	Camera/import flow, crop, perspective correction, filters, reorder and export	Later
Optimize	Compress PDF	Lossless/structural, balanced and strong modes; image DPI/JPEG controls; result comparison	MVP
Optimize	Repair PDF	Normalize malformed structure, rebuild cross-references, salvage readable pages	MVP
Optimize	PDF/A conversion	Archival conversion with conformance validation where supported	Later
Convert to PDF	Images to PDF	JPG, PNG, TIFF, BMP, WebP; page size, margins, orientation and image quality	MVP
Convert to PDF	Office to PDF	DOC/DOCX, XLS/XLSX, PPT/PPTX, ODT/ODS/ODP using LibreOffice headless	MVP
Convert to PDF	HTML/text to PDF	Local HTML, Markdown, TXT and web print workflows with page controls	Later
Convert from PDF	PDF to images	JPG/PNG/TIFF/WebP; page ranges, DPI and per-page ZIP	MVP
Convert from PDF	PDF to Word	Layout-aware extraction; OCR fallback; confidence warnings	Later
Convert from PDF	PDF to Excel	Table detection, page selection, CSV/XLSX output and manual review	Later
Convert from PDF	PDF to PowerPoint	Page-as-slide and editable-content modes	Later
Convert from PDF	PDF to text/Markdown	Reading-order extraction, OCR fallback and cleanup	MVP
Edit	Add text and shapes	Positioned text, rectangles, ellipses, lines, arrows, stamps and opacity	MVP
Edit	Highlight and draw	Highlight, underline, strikethrough, freehand ink and comments	MVP
Edit	Insert image	Place logo, photo, signature or stamp with resize/rotate/crop	MVP
Edit	Add page numbers	Position, format, page range, start number and exclusions	MVP
Edit	Header and footer	Text, date, filename, page count and	Later

ROHITH DOCUMENT STUDIO (RDS) - PRODUCT SPECIFICATION

Area	Tool	Required behavior	Build
		reusable templates	
Edit	Watermark	Text/image watermark; position, scale, angle, opacity and page scope	MVP
Edit	Crop pages	Visible box selection, fixed margins and uniform crop across ranges	MVP
Edit	Redact	Search or rectangle redaction that removes underlying content; optional sanitization	MVP
Edit	Compare PDFs	Side-by-side, overlay and visual-difference report	Later
Forms	Fill forms	Read/write AcroForm text, choice, checkbox, radio and signature fields	MVP
Forms	Create forms	Add fields, validation, required state, tab order and appearance	Later
Forms	Flatten forms	Bake field appearances into final pages	MVP
OCR	Searchable PDF	OCR scanned documents while preserving page images	MVP
OCR	Languages	English, Telugu and Hindi packs first; additional language packs installable	MVP
OCR	Deskew and cleanup	Deskew, rotate, remove background, optimize and detect orientation	MVP
OCR	Text export	TXT, searchable PDF, hOCR/ALTO and page-level confidence	Later
Security	Password protection	User/open password, owner permissions and AES encryption via qpdf	MVP
Security	Unlock PDF	Remove password restrictions only when the user supplies valid credentials	MVP
Security	Metadata control	View/edit/remove title, author, subject, keywords and timestamps	MVP
Security	Sanitize	Remove hidden metadata, embedded files, scripts and selected annotations	Later
Signature	Draw/type/upload signature	Reusable local signature vault protected by OS storage	MVP
Signature	Place signatures	Drag placement, initials, date and multi-page repeat	MVP
Signature	Cryptographic signing	Certificate-based digital signature and validation	Later
Automation	Batch processing	Apply one operation to many files with naming templates and failure isolation	MVP
Automation	Saved workflows	Chain operations such as OCR -> compress -> watermark -> protect	Later
Automation	Watched folders	Automatically run a saved workflow on new files	Later
Automation	CLI	Scriptable commands for all stable operations	Later
AI	Summarize documents	Local/external model adapter; page citations and selectable scope	Later
AI	Ask document	Retrieval over extracted text with source page references	Later
AI	Classify and rename	Suggest document type, filename and destination folder	Later

Area	Tool	Required behavior	Build
AI	Extract structured data	Invoices, receipts, forms and custom schema extraction with review	Later
Platform	Job history	Inputs, settings, outputs, duration, errors and re-run capability	MVP
Platform	Profiles and presets	Reusable quality, page-number, watermark and security settings	MVP
Platform	Print center	Print preview, grayscale, duplex guidance, N-up, booklet and fit-to-page	Later

5. Core User Workflows

Workflow	Happy path
Merge	Drop files -> inspect pages -> reorder -> choose bookmark behavior -> preview -> export new PDF -> verify page count.
Compress	Open PDF -> preflight -> choose mode/custom settings -> estimate -> process -> compare size and quality -> save.
OCR	Open scans -> choose languages and cleanup -> preview representative pages -> OCR -> verify searchability -> export.
Redact	Open -> search/select regions -> preview marked areas -> apply irreversible redaction -> sanitize -> verify content removal.
Convert Office	Select files -> choose PDF options -> run LibreOffice adapter -> inspect warnings -> verify page count -> save.
Batch workflow	Select folder/files -> choose recipe -> preview naming and destinations -> run isolated jobs -> review summary.
Sign	Open -> select signature identity -> place signature/date/initials -> optionally certify -> export new signed copy.

6. Desktop Information Architecture

Navigation	Contents
Home	Recent jobs, quick tools, drop zone and saved workflows.
Organize	Merge, split, extract, remove, reorder and rotate.
Convert	To PDF, from PDF, image, Office, text and archival formats.
Edit	Text, image, annotation, page number, watermark, crop and redact.
OCR	Searchable PDF, language packs and cleanup.
Forms and Sign	Fill, create, flatten, signatures and certificate validation.
Protect	Encrypt, unlock, permissions, metadata and sanitize.
Automate	Batch, recipes, watched folders and CLI examples.
History	Searchable job log with re-run, reveal output and cleanup.
Settings	Dependencies, language packs, storage, privacy, updates and diagnostics.

7. Workspace Layout

- Left rail: product sections and history.
- Center canvas: document viewer or operation-specific file list.
- Thumbnail panel: page selection, reorder and page status.
- Right inspector: operation settings, presets, validation and output naming.
- Bottom job tray: progress, current stage, cancel, retry and result actions.
- Command palette: keyboard access to every tool and recent preset.

8. Technical Architecture

Use a thin desktop shell and a stable local processing layer. The UI never directly executes arbitrary system commands. Each tool calls a typed Rust command, which validates inputs and invokes an adapter in an isolated job directory.

Layer	Technology	Responsibility
Desktop shell	Tauri 2	Small installer, native file permissions, secure IPC and Windows packaging.
UI	React + TypeScript + Vite	Fast component development and strongly typed operation forms.
Viewer	PDF.js	Rendering, thumbnails, selection, text search and coordinates.
Core	Rust	Filesystem safety, IPC, cancellation, job lifecycle and dependency management.
Operation adapters	Rust/Python/CLI wrappers	qpdf, PDF edit library, LibreOffice, OCRmyPDF/Tesseract and image utilities.
Local database	SQLite	Settings, history, presets, workflow definitions and dependency status.
Temporary storage	Per-job directories	Inputs copied or referenced safely; outputs staged before final move.
Secrets	OS credential store	Protect certificate passwords, API keys and signature-vault secrets.
Updates	Signed application updates	Optional; verify publisher/signature before installation.

9. Suggested Processing Stack

Capability	Preferred engine	Notes
PDF structure	qpdf	Merge/split/select pages, encryption, decryption, linearization, structural checks and repair.
PDF rendering/editing	PyMuPDF or a Rust PDF library	Render previews, extract text/images, overlay content, annotations, crop and compare. Confirm license compatibility before distribution.
Office conversion	LibreOffice headless	DOCX/XLSX/PPTX/ODF to PDF and supported reverse/export workflows.
OCR	OCRmyPDF + Tesseract	Searchable PDFs, skew, rotate, cleanup, language packs and optimization.
Image	ImageMagick/libvips or Rust image	Format conversion, resize, DPI and image optimization.
PDF viewer	PDF.js	Interactive rendering and text layer.
Digital signature	Standards-compliant signing library	CMS/PAdES signing and verification; milestone after basic signature placement.
AI	Provider-neutral adapter	Local model server first; external endpoint only per-job opt in.

10. Unified Operation Contract

Every tool must implement the same lifecycle so Codex can add capabilities without inventing a new pattern each time.

Field	Requirement
id	Stable operation identifier, for example pdf.merge.
input schema	Accepted file types, multiplicity, size checks, passwords and page-selection syntax.
settings schema	Typed defaults, validation ranges and preset serialization.
preflight	Dependency checks, file readability, encryption state, expected output type and risk warnings.
estimate	Optional page count, expected work units and approximate output-size range.
execute	Cancellable worker with structured progress events.
verify	Openability, page count, encryption/signature/OCR checks and expected output constraints.
cleanup	Delete temporary files on success, failure, cancel and crash recovery.
audit record	Input fingerprints, settings, versions, timings, output and error details without storing document content.

11. Job State Machine

QUEUED -> PREFLIGHT -> READY -> RUNNING -> VERIFYING -> COMPLETED. Alternate terminal states: FAILED or CANCELLED. A crash-recovery scan marks abandoned RUNNING jobs as INTERRUPTED, cleans unsafe temporary files and offers retry when inputs still exist.

12. Repository Structure

```

rds/
  apps/desktop/      # Tauri + React application
  crates/core/       # job model, operation traits, filesystem safety
  crates/pdf_ops/    # qpdf and PDF editing adapters
  crates/office_ops/ # LibreOffice adapter
  crates/ocr_ops/    # OCRmyPDF and Tesseract adapter
  crates/security/   # secret storage, certificates, sanitization
  packages/ui/       # reusable React components
  packages/contracts/ # shared TypeScript schemas
  operations/        # declarative operation manifests
  tests/fixtures/    # licensed/synthetic PDFs and expected outputs
  tests/integration/ # operation and pipeline tests
  scripts/           # dependency install, packaging and diagnostics
  docs/              # architecture decisions and user documentation

```

13. Functional Requirements

ID	Requirement
FR-001	The application shall accept drag-and-drop files and folders where the selected operation supports them.
FR-002	The application shall reject unsupported or unreadable inputs before starting a worker.
FR-003	The application shall preserve originals unless an explicit replace-original action is confirmed.
FR-004	The application shall support page selection using thumbnails and range syntax.
FR-005	The application shall provide progress, current stage and cancellation for long jobs.
FR-006	The application shall verify every completed output before showing success.
FR-007	The application shall isolate failures in multi-file jobs and continue when safe.
FR-008	The application shall store reusable presets and saved workflows locally.
FR-009	The application shall expose all stable operations through a consistent local API and later CLI.
FR-010	The application shall allow users to inspect, open, reveal, rerun and remove history entries.
FR-011	The application shall detect missing dependencies and provide an exact remediation path.
FR-012	The application shall support English initially and keep UI strings ready for localization.
FR-013	OCR shall allow multiple languages in one job and report page-level failures.
FR-014	Redaction shall remove underlying text/image content, not merely draw a black rectangle.
FR-015	Protection shall clearly distinguish opening password from permissions password.
FR-016	AI operations shall identify whether processing is local or external before execution.
FR-017	External AI shall never receive the file unless the user explicitly approves that job.
FR-018	The app shall display source-page references for AI answers derived from a document.
FR-019	The app shall support deterministic output naming templates with collision handling.
FR-020	The app shall provide a diagnostics export that excludes document contents and secrets.

14. Non-Functional Requirements

ID	Requirement
NFR-001 Performance	Open a 100-page normal PDF preview within 5 seconds on the target laptop, excluding damaged files.
NFR-002 Responsiveness	UI interactions remain responsive while workers run; heavy operations execute outside the UI thread.
NFR-003 Reliability	No completed job is reported as successful until the output passes verification.
NFR-004 Privacy	No telemetry, uploads or external calls are enabled by default.
NFR-005 Security	Validate paths, quote arguments safely, use allow-listed executables and isolate temporary directories.
NFR-006 Recovery	Interrupted jobs are detected on next launch and temporary data is safely reconciled.
NFR-007 Accessibility	Keyboard navigation, visible focus, screen-reader labels and 200% scaling without blocked controls.
NFR-008 Maintainability	Each operation has a manifest, typed settings, adapter, tests and documentation.
NFR-009 Portability	Core contracts avoid Windows-only assumptions even though Windows is the first package target.
NFR-010 Print quality	Generated reports and print workflows support A4, grayscale and readable minimum font sizes.

15. Privacy and Security Model

- Default network posture: no network request is required for document processing.
- Temporary files: private per-job directories, predictable cleanup and startup recovery.
- File fingerprints: store path, size, modification time and optional hash; do not store extracted text unless a feature requires it and the user enables it.
- Command execution: no shell string concatenation; pass validated argument arrays to allow-listed binaries.
- Untrusted PDFs: treat parsers and converters as attack surfaces; maintain patched dependencies and use restricted worker permissions where feasible.
- Secrets: use Windows Credential Manager/OS keychain; never write API keys or certificate passwords to logs.
- External processing: show provider, data scope and retention warning before each first use; maintain a global disable switch.
- Clean-room boundary: build original source code and UI. Do not copy proprietary assets, documentation, terms, hidden APIs or implementation code.

16. Data Model

Entity	Important fields
Job	id, operation_id, state, created_at, started_at, finished_at, progress, current_stage, error_code, error_detail
JobInput	job_id, path, display_name, mime, size, page_count, fingerprint, password_reference
JobOutput	job_id, path, mime, size, page_count, verification_status, warnings
Preset	id, operation_id, name, settings_json, created_at, updated_at
Workflow	id, name, ordered_steps, variable mappings, naming template, destination policy

Entity	Important fields
Dependency	id, detected_version, path, status, last_checked, capabilities
AppSetting	key, value, scope
SignatureIdentity	id, label, local_asset_reference, certificate_reference, protected metadata

17. Build Roadmap

Phase	Scope	Exit condition
0. Foundation	Tauri shell, React design system, SQLite, secure paths, operation contract, job tray, dependency diagnostics and test fixtures.	App launches, opens a PDF and runs a dummy cancellable verified job.
1. Core PDF	Merge, split, extract, remove, reorder, rotate, compress, repair and PDF-to-images.	All core operations pass golden-file and failure tests.
2. Conversion	Images-to-PDF, Office-to-PDF, PDF-to-text, output naming and batch runner.	Mixed batch produces verified outputs and useful summary.
3. Workspace editing	Viewer, annotations, text/image overlay, page numbers, watermark, crop and flatten.	Visual regression confirms coordinates and render parity.
4. OCR and security	OCR languages, skew, searchable PDF, encrypt, unlock, metadata, sanitize and true redaction.	Search and extraction work; redacted content cannot be recovered by text extraction.
5. Forms and signing	Fill/flatten forms, reusable signature placement, certificate signing and verification.	Outputs validate in independent viewers.
6. Automation	Saved workflows, watched folders, CLI and reusable presets.	Restart-safe workflows with collision and error handling.
7. AI assistance	Summaries, Q&A, classification, rename and structured extraction with local/external adapters.	Answers cite pages; external transmission requires consent.
8. Polish	Installer, signed updates, localization, accessibility, performance and recovery hardening.	Release checklist passes on clean Windows machine.

18. Codex Work Rules

1. Work in small vertical slices. Each slice includes UI, typed command, adapter, verification, tests and documentation.
2. Do not implement many tools as independent ad-hoc scripts. Add them through the shared operation contract.
3. Never report success based only on process exit code. Reopen and verify the output.
4. Use synthetic or user-owned test documents. Keep fixtures small and documented.
5. Do not silently weaken a feature. For example, a redaction overlay is not accepted as true redaction.
6. Record architecture decisions for libraries, licensing, coordinate systems, passwords, signatures and OCR behavior.
7. Keep destructive actions behind explicit confirmation and retain undo until export where practical.
8. Run formatting, unit tests, integration tests and package smoke tests before marking a task complete.

19. Test Strategy

Test layer	Coverage
Unit	Range parser, naming templates, schemas, path validation, job transitions and settings migration.
Adapter integration	Known input -> operation -> verified output, including encrypted, malformed, scanned and large documents.
Golden visual	Render selected outputs and compare images with tolerance.
Security	Path traversal, command injection, malformed PDFs, secret redaction in logs and cleanup after cancel/crash.
Cross-viewer	Open outputs in PDF.js, Chromium and at least one independent desktop viewer.
Performance	10, 100, 500 and 1,000-page files; image-heavy, OCR and large-batch scenarios.

Test layer	Coverage
Accessibility	Keyboard-only workflow, focus order, labels, zoom and high-contrast Windows settings.
Installer	Fresh Windows VM, upgrade, uninstall, missing dependencies and repair installation.
Recovery	Kill process during each job stage, relaunch and verify no false success or orphaned unsafe state.

20. Definition of Done for Any Tool

- Operation manifest and typed settings are complete.
- UI explains accepted inputs and risky settings.
- Preflight catches common failures before processing.
- Progress and cancellation work.
- Output is written to a staging location and verified.
- Original file remains unchanged by default.
- Temporary files are cleaned after success, failure and cancellation.
- Unit and integration tests cover success and at least three meaningful failures.
- History entry contains actionable diagnostics but no secret or document content.
- User documentation includes one practical example.

21. Initial Acceptance Test Pack

Test	Expected result
AT-01 Merge	Merge 3 PDFs with 2, 5 and 1 pages; reorder; output has 8 pages in requested order.
AT-02 Split	Split a 10-page PDF into 1-3, 4-7 and 8-10; outputs have 3, 4 and 3 pages.
AT-03 Compress	Compress image-heavy PDF; output opens, page count matches and size decreases without unreadable text.
AT-04 Office	Convert DOCX/XLSX/PPTX samples; outputs open and warnings are shown for unsupported content.
AT-05 OCR	OCR English+Telugu scan; output text is searchable and original page appearance is retained.
AT-06 Redact	Redact selected words; extraction and copy/paste cannot recover them.
AT-07 Protect	Encrypt with open password; wrong password fails and correct password opens.
AT-08 Cancel	Cancel OCR midway; no completed output is published and temp data is cleaned.
AT-09 Crash	Terminate during merge; relaunch detects interruption and does not claim success.
AT-10 Batch	Process 20 mixed files with 2 invalid inputs; 18 succeed, 2 fail independently with reasons.

22. Master Prompt for Codex

Build “Rohith Document Studio (RDS)”, a Windows-first, offline-first personal document and PDF workspace.

Technology:

- Tauri 2 desktop shell
- React + TypeScript + Vite UI
- Rust core for secure IPC, job lifecycle, paths, cancellation and dependency execution
- PDF.js viewer
- SQLite for local settings, presets, workflows and job history
- Adapter layer for qpdf, a suitable PDF editing/rendering library, LibreOffice headless, OCRmyPDF/Tesseract and image conversion

Non-negotiable rules:

1. All implemented features are included in one complete edition; no subscription or artificial limits.
2. Files remain local by default. No telemetry or uploads by default.
3. Never overwrite originals without explicit confirmation.
4. Every operation must implement the shared operation contract: schema, preflight, estimate, execute, progress, cancel, verify, cleanup and audit record.
5. Never show success until the output is reopened and verified.
6. Use original code, copy, assets and interface design. Implement functional requirements cleanly; do not copy proprietary code or branding.
7. Start with Phase 0 only. Do not jump directly into the full feature list.

Phase 0 deliverables:

- Monorepo structure defined in the specification
- Tauri app with left navigation, home drop zone, central PDF viewer placeholder, right inspector and bottom job tray
- SQLite migrations and typed Job/JobInput/JobOutput/Preset models
- Rust operation trait and one “diagnostic.copy” reference operation that supports preflight, progress, cancel, verify and cleanup
- Dependency diagnostics screen
- Secure per-job temporary directories and startup recovery
- Unit/integration tests and a Windows development setup document

Work method:

- First inspect the repository and write a concise implementation plan.
- Make one coherent change at a time.
- Run formatter, typecheck, tests and a package/dev smoke test after each slice.
- Document assumptions and unresolved library/licensing decisions.
- Stop after Phase 0 is fully working and provide exact commands, test results and next recommended slice.

23. Recommended First Real Feature Slice

After Phase 0, implement PDF merge as the first production tool because it exercises multi-file input, page counting, reorder settings, qpdf execution, progress, output naming, verification, cleanup and history without requiring complex coordinate editing.

Slice task	Completion evidence
Input	Multiple PDFs accepted; encrypted and unreadable files identified in preflight.
Settings	Drag reorder, whole-file/range selection, bookmarks option and output name.
Execution	Cancellable qpdf adapter using validated argument arrays.
Verification	Output reopens, expected page count and page ordering verified.
Failure	One corrupt/encrypted input produces a specific actionable error.
Tests	Golden merge test, cancel test, naming collision test and crash cleanup test.
UX	Preview list, progress stages and open/reveal result actions.

24. Naming and Brand Direction

Recommended name: Rohith Document Studio. Short name: RDS. It is personal, broad enough for non-PDF formats and avoids implying that the product is merely a clone of another service. Suggested descriptor: "Your private document workshop."

Element	Recommendation
Name	Rohith Document Studio
Short name	RDS
Descriptor	Your private document workshop
Visual direction	Original monochrome-first interface with simple document, page and tool icons
File extension for workflows	.rdsflow (JSON internally)
Executable	RohithDocumentStudio.exe
Repository	rohith-document-studio

25. Reference Implementation Notes

The following primary project documentation should be used during implementation and version selection: qpdf documentation for structural PDF operations and encryption; OCRmyPDF/Tesseract documentation for OCR; LibreOffice documentation for headless Office conversion; PDF.js documentation for viewing; Tauri documentation for desktop security and permissions. Library versions and licenses must be confirmed at build time.

26. Final Recommendation

Start building RDS as a private offline desktop tool, not as a web service. Complete the foundation and one verified merge workflow before expanding. This reduces privacy risk, hosting complexity and scope while establishing an architecture that can later support every catalogue item without rewriting the application.